

Unit-II: Server-Side Programming with Python (8 hours)

Compiled By

Lecturer, Sunil Bahadur Bist

NAST, Dhangadhi

Setting up the Python environment

- Installing Python

Download and Install Python from the official website

<https://www.python.org/downloads/>

- Find the Python Version & Installation Directory

`python --version`

where python

- To Install python Packages

`pip install <<packagename>>`

- To list installed python packages

`pip list`

Using a virtual environment

- To install virtual environment

```
pip install virtualenv
```

- To Check the version of virtual environment

```
virtualenv --version
```

- To Create your own virtual environment

```
virtualenv <env_name>>
```

OR

```
python -m venv <<myenv>>
```

Using a virtual environment...

- Activate Virtual Environment on Windows

```
cd <env_name>
```

```
Scripts\activate
```

- Deactivate Virtual Environment

```
deactivate
```

(Note: Remember to activate the relevant virtual environment every time you work on the project)

Choose a Code Editor/IDE

- Install VSCode

Download and Install VSCode and Python Plugin

<https://code.visualstudio.com/>

- PyCharm IDE

Download and Install PyCharm Community Python IDE

<https://www.jetbrains.com/pycharm/>

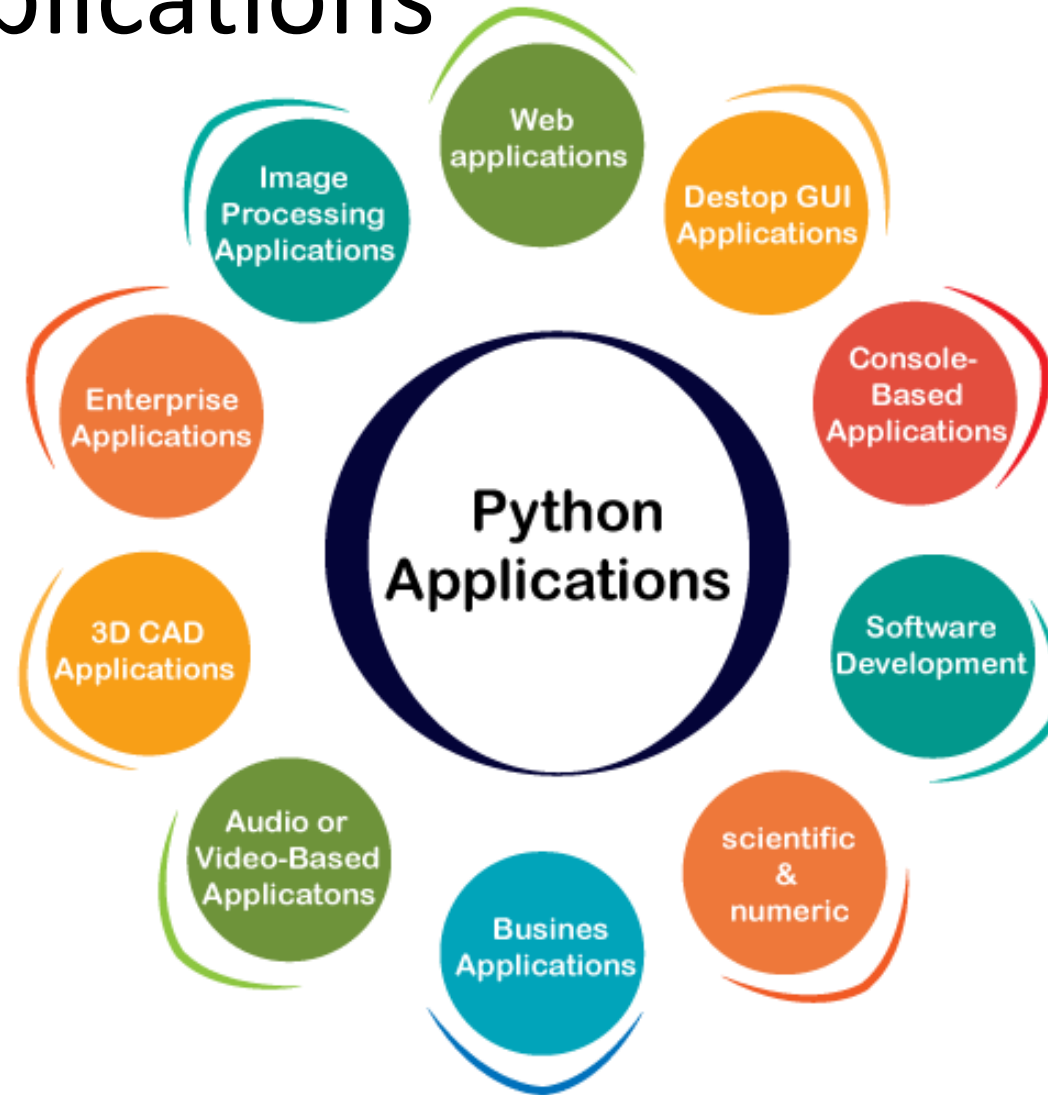
Introduction to Python

- Python is a popular programming language.
- It was created by Guido van Rossum, and released in 1991.
- Python programs are executed by an interpreter.
- There are many different environments in which the Python interpreter might run an IDE, a browser, or a terminal window.
- It is a versatile programming language used for a wide range of applications, including web development, software development, data analysis, machine learning, and automation.

Why Python?

- Python works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc).
- Python has a simple syntax similar to the English language.
- Python has syntax that allows developers to write programs with fewer lines than some other programming languages.
- Python runs on an interpreter system, meaning that code can be executed as soon as it is written. This means that prototyping can be very quick.
- Python can be treated in a procedural way, an object-oriented way or a functional way.

Python Applications



Python syntax

- Python was designed for readability, and has some similarities to the English language with influence from mathematics.
- Python uses new lines to complete a command, as opposed to other programming languages which often use semicolons or parentheses.
- Python relies on indentation, using whitespace, to define scope; such as the scope of loops, functions and classes. Other programming languages often use curly-brackets for this purpose.

Python first program

```
print("Hello, World!")
```

Save as hello.py

To run the program

```
python hello.py
```

The Python Command Line

- Type `py` or `python` in command prompt
- When the interpreter starts, a prompt appears where you can type programs into a so-called “read-evaluation-print loop” (or REPL).
- For example, in the following output, the interpreter displays its copyright message and presents the user with the `>>>` prompt, at which the user types a familiar “Hello World” program:

```
>>> print('Hello World')
```

```
Hello World
```

Python Comments

- Comments in python can be used to explain Python code.
- It can be used to make the code more readable.
- It can be used to prevent execution when testing code.
- It starts with a #, and Python will ignore them

For example,

```
#This is a simple single line comment
```

```
print("Hello, World!")
```

Python Comments

- Python does not really have a syntax for multiline comments.
- To add a multiline comment you could insert a **# for each line:**

For example,

```
#This is a comment
```

```
#written in
```

```
#more than just one line
```

```
print("Hello, World!")
```

OR

Python Comments

```
"""This is a comment  
written in M  
more than just one line  
""">  
print("Hello, World!")
```

Or

```
""" Comment in  
Multiline  
multiple  
line """
```

As long as the string is not assigned to a variable, Python will read the code, but then ignore it, and you have made a multiline comment.

Basic operations

- Python has several types of operators to perform different operations:

- Arithmetic operators:

- Used for mathematical calculations.

+ (addition)

- (subtraction)

* (multiplication)

/ (division)

// (floor division - returns the integer part of the division)

% (modulus - returns the remainder of the division)

** (exponentiation)

Basic operations

- Assignment operators:
- Used to assign values to variables.

= (assigns the value on the right to the variable on the left)

+=, -=, *=, /=, //=, %=, **=

(compound assignment operators, combine an arithmetic operation with assignment)

Basic operations

- Comparison operators:
- Used to compare values.

== (equal to)

!= (not equal to)

> (greater than)

< (less than)

>= (greater than or equal to)

<= (less than or equal to)

Basic operations

- Logical operators:
- Used to combine or modify boolean values.
 - and (returns True if both operands are True)
 - or (returns True if at least one operand is True)
 - not (returns the opposite boolean value of the operand)

Basic operations

- Bitwise operators:
- Used to perform operations on individual bits.

& (AND)

| (OR)

^ (XOR)

~ (NOT)

<< (left shift)

>> (right shift)

Basic operations

- Membership operators:
- Used to check if a value is present in a sequence (e.g., list, tuple, string).

`in` (returns True if the value is present)

`not in` (returns True if the value is not present)

Basic operations

- Identity operators:
- Used to compare the identity of objects (i.e., if they are the same object in memory).

is (returns True if both operands are the same object)

is not (returns True if both operands are not the same object)

Basic operations

- The order of precedence of operators in Python (from highest to lowest) is:
- Parentheses, Exponentiation, Multiplication, Division, Floor division, Modulus, Addition, Subtraction, Comparison operators, Logical operators, and Assignment operators.

Data Types and Variables

- Python has several built-in data types used to classify data.
- Variables are names assigned to stored data values, acting as references to memory locations.

Data Types

- Numeric Types:
- int: Represents whole numbers (e.g., 10, -5, 0).
- float: Represents real numbers with decimal points (e.g., 3.14, -2.5).
- complex: Represents complex numbers in the form $a + bj$, where a and b are floats and j is the imaginary unit (e.g., $2 + 3j$).

Data Types

String Type:

- str: Represents a sequence of characters, used for text (e.g., "hello", 'world').

Sequence Types:

- list: An ordered, mutable (changeable) collection of items (e.g., [1, 2, 3], ['a', 'b', 'c']).
- tuple: An ordered, immutable (unchangeable) collection of items (e.g., (1, 2, 3), ('a', 'b', 'c')).
- range: Generates a sequence of numbers within a specified range.

Data Types

Mapping Type:

- dict: A collection of key-value pairs, where keys are unique and immutable (e.g., {'name': 'Alice', 'age': 30}).

Set Types:

- set: An unordered collection of unique items (e.g., {1, 2, 3}).
- frozenset: An immutable version of a set.

Boolean Type:

- bool: Represents truth values, either True or False.

Data Types

Binary Types:

- `bytes`: Represents immutable sequences of bytes.
- `bytearray`: Represents mutable sequences of bytes.
- `memoryview`: Provides a memory view of an object without copying.

None Type:

- `NoneType`: Represents the absence of a value (`None`).

Variables

- Variables are created when a value is assigned to them.
- Python is dynamically typed, meaning the data type of a variable is automatically inferred at runtime and can be changed.
- For example,

```
x = 10 # x is an integer
```

```
x = "hello" # x is now a string
```

Variables

- Variable names must follow certain rules:
- They can contain letters, numbers, and underscores, but cannot start with a number.
- They are also case-sensitive.
- For example,

`my_variable = 10`

`_my_variable = 20`

`myVariable = 30`

Variables

- If you want to specify the data type of a variable, this can be done with casting.

```
x = str(3)  # x will be '3'
```

```
y = int(3)  # y will be 3
```

```
z = float(3) # z will be 3.0
```

- Checking Data Types
- The `type()` function is used to determine the data type of a variable.
- For example,

```
x = 10
```

```
print(type(x)) # Output: <class 'int'>
```

Variable Assignment & Naming Conventions

- Variable assignment in Python involves associating a name with a value, allowing the program to store and manipulate data.
- The assignment is done using the equals sign (=).
- For example, `x = 10` assigns the value 10 to the variable x.
- Python is dynamically typed, so you don't need to explicitly declare the variable type; it's inferred from the assigned value.

Variable Assignment

Basic Assignment

- For example,

name = "Alice"

age = 30

price = 99.99

is_active = True

Variable Assignment

Multiple Assignment

- Python allows assigning the same value to multiple variables in a single line or assigning multiple values to multiple variables.
- For example,

`a = b = c = 100`

`x, y, z = 1, 2, 3`

Variable Assignment

Reassignment

- A variable can be reassigned to a new value, even of a different type.
- For example,

`x = 5`

`x = "Hello"`

Variable Naming Rules

- Variable names can contain letters, numbers, and underscores.
- They must start with a letter or an underscore.
- They are case-sensitive (myVar and myvar are different).

Unpacking

- Python allows unpacking values from collections (like lists or tuples) into variables.

```
fruits = ["apple", "banana", "cherry"]
```

```
x, y, z = fruits
```

Variable Naming Rules

Deleting variables

- A variable can be deleted using the del keyword

```
x = 10
```

```
del x
```

- # trying to print(x) here will raise an error since x does not exist anymore

Naming Conventions

- Python naming conventions, as outlined in PEP 8, aim to improve code readability and consistency.
- PEP stands for Python Enhancement Proposal.
- Here's a summary:

Modules and Packages:

- Short, lowercase names.
- Underscores can be used if it improves readability (e.g., `my_module`, `my_package`).

Naming Conventions

Classes:

- Use CamelCase (e.g., MyClass, HTTPServer).
- Functions and Methods:
- Lowercase with words separated by underscores (snake_case) (e.g., calculate_area, get_name).

Variables:

- Lowercase with words separated by underscores (snake_case) (e.g., user_name, total_count).

Constants:

- All uppercase with words separated by underscores (e.g., MAX_SIZE, API_KEY).

Naming Conventions

Avoid:

- Single-character names (except for counters or iterators).
- Reserved keywords.
- Names that are too general or too verbose.

Be Descriptive:

- Choose names that clearly indicate the purpose of the variable, function, or class.

Naming Conventions

Case Sensitivity:

- Python is case-sensitive, so `myVar` and `myvar` are different variables.

Underscores:

- Leading underscores (e.g., `_my_var`) suggest internal use.
- Double leading underscores (e.g., `__my_var`) name mangling for class-specific variables.
- Trailing underscores (e.g., `my_var_`) are sometimes used to avoid conflicts with Python keywords.

Acronyms:

- When using acronyms in CamelCase names, capitalize all letters of the acronym (e.g., `HTTPServer` instead of `HttpServer`).

Type Conversion

Type conversion, also known as type casting, involves changing a variable's data type from one type to another.

Python supports two main types of conversion:

- implicit and
- explicit.

Implicit Type Conversion

- Python automatically converts one data type to another without the need for explicit user intervention.
- This usually happens when performing operations involving different data types.
- For example, when adding an integer and a float, Python will implicitly convert the integer to a float before performing the addition.

Implicit Type Conversion

- For example,
 `a_int = 10`
 `b_float = 5.5`
 `result = a_int + b_float`
 `print(result)`
 `print(type(result))`

Output:

- 15.5
- <class 'float'>

Explicit Type Conversion

- Users manually convert the data type of an object using built-in functions.
- This is useful when you need to convert a value to a specific type for further operations or to meet certain requirements.
 - `int()`: Converts a value to an integer data type.
 - `float()`: Converts a value to a floating-point number.
 - `str()`: Converts a value to a string.
 - `bool()`: Converts a value to a boolean (True or False).
 - `list()`, `tuple()`, `set()`: Convert to list, tuple and set types respectively.

Explicit Type Conversion

- For example,

```
num_str = "123"
num_int = int(num_str)
print(num_int)
print(type(num_int))
price_str = "99.99"
price_float = float(price_str)
print(price_float)
print(type(price_float))
age = 30
age_str = str(age)
```

```
print(age_str)
print(type(age_str))
```

Output:

```
123
<class 'int'>
99.99
<class 'float'>
30
<class 'str'>
```